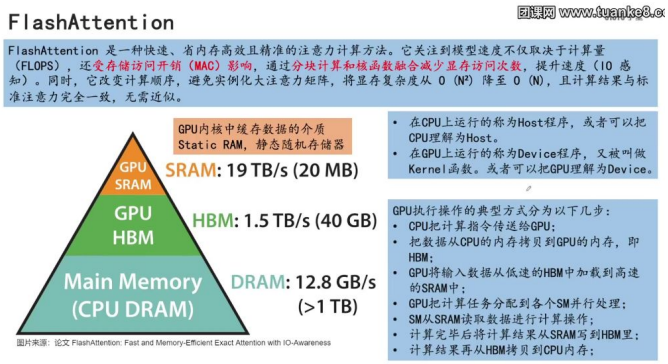
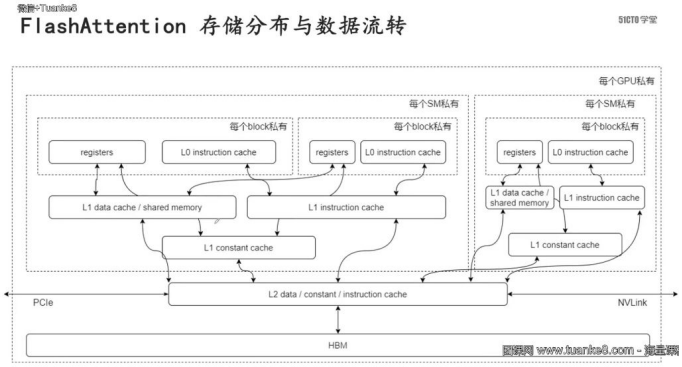
以下为AI生成的图文笔记的内容

一、FlashAttention 00:00

1. 问题引入 00:23

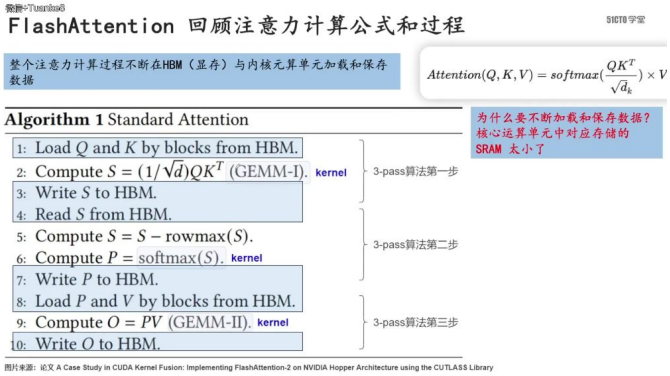


-
- **核心优势**:　快速、省内存且精准的注意力计算方法，通过优化存储访问提升10倍速度感知
- **关键创新**:
  - 采用分块计算(tiling)将大矩阵切分为小块
  - 核函数融合(kernel fusion)减少显存访问次数
  - 改变计算顺序避免实例化大注意力矩阵
- **显存优化**: 将复杂度从$O(N^2)$降至O(N)，同时保持计算结果与标准注意力完全一致



-
- **存储介质对比**:
  - **SRAM**: 19TB/s带宽但容量仅20MB（GPU内核缓存）
  - **HBM**: 1.5TB/s带宽40GB容量（GPU显存）
  - **DRAM**: 12.8GB/s带宽（CPU主内存）
- **数据流转瓶颈**:
  - 计算过程需在HBM与SRAM间多次往返传输
  - 典型流程：CPU指令→HBM→SRAM→SM计算→SRAM→HBM→CPU内存

2. 回顾注意力计算公式和过程 06:08



-
- **计算公式**:$Attention(Q,K,V) = softmax(\frac{QK^T}{\sqrt{d_k}}) \times V$
- **传统计算缺陷**:
  - **三次显存往返**:
    - 加载Q,K→计算$S = \frac{1}{\sqrt{d}}QK^T$→写回HBM
    - 读取S→计算$P = softmax(S)$→写回HBM
    - 加载P,V→计算$O = PV$→写回HBM
  - **效率低下原因**: SRAM容量限制导致中间结果必须存回HBM
- **优化方案**:
  - **分块计算**: 将Q,K,V矩阵切分为适合SRAM的小块
  - **核函数融合**: 在SRAM内完成全部计算步骤，避免中间结果写回
  - **计算顺序重构**: 采用增量式计算策略，仅保留必要中间状态

二、知识小结

知识点	核心内容	技术难点/关键点	重要性
KV缓存优化	通过方法降低多头注意力机制中KV缓存占用，减轻GPU数据加载压力	序列增长导致QKV向量变长，GPU加载效率下降	★★★★
Flash Attention原理	快速高效的内存节省注意力计算方法	**分块计算**（切分大矩阵）和**核函数融合**（运算单元内完成计算）	★★★★★
GPU内存架构	三级存储结构: - 主内存（CPU） - 显存HBM（GPU） - SRAM（GPU内核缓存）	数据需经多次传输: 主内存→显存→SRAM→计算→回写	★★★
传统Attention计算瓶颈	公式: $softmax(\frac{QK^T}{\sqrt{d}})V$	**显存带宽成为性能瓶颈**	★★★★

	需3次显存读写： 1. 加载QK→计算→写回 2. 加载→softmax→写回 3. 加载PV→计算→写回	中间结果频繁IO操作	
Flash Attention优化策略	1. **分块计算**：将大矩阵切分为SRAM可容纳的小块 2. **延迟写回**：中间结果保留在SRAM，最终一次性写回显存	SRAM容量小但速度快，需精确控制分块大小	★★★★★
GPU计算流程对比	传统流程：显存↔SRAM多次往返 优化流程：SRAM内部完成全计算链	**核函数融合**消除冗余数据传输	★★★★